

AI Agents HR System

This entire application is an automated, highly efficient Human Resources department.

- **CrewAI** is the **company's philosophy and training manual**. It provides the framework to "hire" and define the roles for your digital employees (the Agents). It gives them a `role`, a `goal`, a `backstory`, and the ability to use tools.
- The **Agents** (`Orchestrator`, `Matching`, etc.) are the specialist **employees** in your HR department. Each has one specific job they are an expert at.
- The **Tools** (`DatabaseTool`, `EmailTool`, etc.) are the **equipment and software** these employees use to do their jobs (like a computer, a phone, or access to the company's file cabinets).
- **MCP (Master Control Protocol)** is the company's **chain of command and workflow protocol**. It dictates that the `OrchestratorAgent` is the **manager**, and all other agents are subordinates who receive tasks and report back. This ensures a structured and predictable process.
- **Databases (MongoDB & FAISS)** are the department's **filing systems**. MongoDB is the highly organized filing cabinet for factual records, and FAISS is the "concept library" for understanding the meaning of resumes.

The Flowchart Explained: A Candidate's Journey

Let's follow a single resume from submission to the final decision, as depicted in your flowchart.

Phase 1: The Application (Ingestion Layer)

This is how a resume enters your system.

1. **The Candidate Submits:** A candidate, our user, finds your job posting. They have two ways to apply:
 - **Google Form:** They fill out their name, email, and upload their resume.
 - **Manual Upload:** They go to your web application and use the "Upload Resume" button.
2. **The Data is Sent to the Backend:**
 - **Google Form Path:** When the form is submitted, the attached **Google Apps Script** automatically triggers. It grabs the resume file from your Google Drive, bundles it with

the candidate's name and email, and sends it to a special API endpoint on your backend (`/forms/submit-application`). This is the automated, "hands-off" route.

- **Manual Path:** The JavaScript on your frontend sends the resume directly to the `/upload-resume/` endpoint.
- 3. **The Backend Receives the Application:** Both paths lead to your **FastAPI Backend Server**. The server receives the data and, to ensure it stays fast and responsive, it immediately hands off the processing task. It says to the `OrchestratorAgent`, "A new candidate has arrived. Here is the path to their resume file. Begin the process."

Phase 2: The Triage (Parsing & Initial Analysis)

Now the `OrchestratorAgent`, the manager of our HR department, takes charge. Its entire job is to delegate tasks in the correct order. This is the **MCP** in action.

1. Task 1: "Understand this Resume"

- The `Orchestrator` delegates the first task to the `ResumeParsingAgent`.
- **Function:** This agent is a specialist in reading and understanding documents.
- **Tools Used:**
 - It uses a `ResumeParserTool` (which internally uses the **LLM on Groq**) to read the raw text and extract structured information like name, email, phone number, and a list of skills.
 - It uses an `EmbeddingTool` to convert the entire resume text into a **resume vector** (a list of numbers representing its meaning).
-
- **Output:** The agent saves the structured data (name, email) to **MongoDB**. It saves the resume vector to the **FAISS** index for future similarity searches. It then reports back to the `Orchestrator` that the resume has been successfully parsed.

2. Task 2: "Confirm Receipt"

- The `Orchestrator` now delegates to the `CommunicationAgent`.
- **Function:** This agent is the department's secretary, responsible for all external communication.
- **Tools Used:** It uses an `EmailTool` to send a pre-defined "Application Received" email to the candidate.

Phase 3: The Big Decision (Matching & Shortlisting)

This is the most critical screening step.

1. Task 3: "Is this Candidate a Good Fit?"

- The `Orchestrator` delegates the most important analysis to the `MatchingAgent`.
- **Function:** This agent is your star recruiter. It doesn't just look for keywords; it deeply understands both the candidate and the job roles.
- **Tools Used:**
 - It first uses the `DatabaseTool` to fetch the candidate's parsed resume data from **MongoDB**.
 - Crucially, it uses the `DatabaseTool` again to fetch **ALL** available job postings from MongoDB.
 - Its most powerful tool is the **LLM (Groq)**. The agent constructs a detailed prompt: *"You are an expert recruiter. Here is a candidate's resume and here are all our open jobs. Analyze them and return a JSON object telling me which job is the single best match, why, and a score from 0-100."*
- **Output:** The LLM returns the detailed JSON. The `MatchingAgent` then uses the `DatabaseTool` one last time to *update* the candidate's record in MongoDB with the final match score. It then reports this score back to the `Orchestrator`.

2. The Orchestrator's Decision:

- The `Orchestrator` receives the score (e.g., 92). It checks this against its rule: `if score >= 50`.
- **If the score is low (< 50):** It proceeds down the "Reject" path, telling the `CommunicationAgent` to send a rejection email. The process ends here for this candidate.
- **If the score is high (>= 50):** It proceeds down the "Shortlist" path.

Phase 4: The AI Interview

This is the automated evaluation stage.

1. Task 4: "Schedule the AI Interview"

- The `Orchestrator` generates a unique interview URL.

- It uses the `DatabaseTool` to create a new document in the `interviews` collection in **MongoDB**. This record includes the candidate's email, the job ID, the unique link, and a status of `pending_ai_interview`.
- It then tells the `CommunicationAgent` to send the "Interview Invitation" email containing the unique link.

2. The Candidate Experience:

- The candidate receives the email and clicks the link, opening the **Frontend Interview Room**.
- The JavaScript on this page immediately establishes a **WebSocket connection** to the backend server.

3. The Interview Begins:

- The WebSocket connection activates the `InterviewAgent` on the backend.
- **Function:** This agent is your automated interviewer. Its job is to conduct a live, conversational interview.
- **Tools Used & Flow:**
 1. It uses the `DatabaseTool` to get the job details so it knows what to ask about.
 2. It uses the **LLM (Groq)** to generate a creative opening question.
 3. The question text is sent over the WebSocket to the frontend. The browser's speech synthesis API speaks it out loud.
 4. The frontend records the candidate's spoken answer and sends the audio blob back over the WebSocket.
 5. The backend receives the audio and uses the **Speech-to-Text (Whisper)** tool to transcribe it into text.
 6. The `InterviewAgent` now takes the transcript of the answer, the conversation history, and the job details, and sends them to the **LLM** with a prompt like, *"Here's the conversation so far. Based on the candidate's last answer, what is the next logical follow-up question?"*
 7. This loop repeats for a set number of questions.

Phase 5: The Final Verdict

1. Task 5: "Evaluate the Interview"

- Once the question loop is over, the `InterviewAgent` takes the *entire conversation transcript*.
- It sends it to the **LLM** with one final, crucial prompt: *"You are a hiring manager. Read this transcript and return a JSON object with a summary, strengths, weaknesses, and a final score from 0 to 100."*
- The agent receives the JSON evaluation and uses the `DatabaseTool` to **update the interview record in MongoDB**, setting the status to `completed_ai_interview` and saving the score and summary.

2. Task 6: "Make the Final Hire/Reject Decision"

- The `InterviewAgent` notifies the `OrchestratorAgent` that the process is complete.
- The `Orchestrator` uses the `DatabaseTool` to read the final `interview_score`.
- It checks its final rule: `if interview_score >= 70`.
- Based on the result, it gives its final command to the `CommunicationAgent`: send either the "Congratulations" email or the final "Thank you for your time" rejection email.